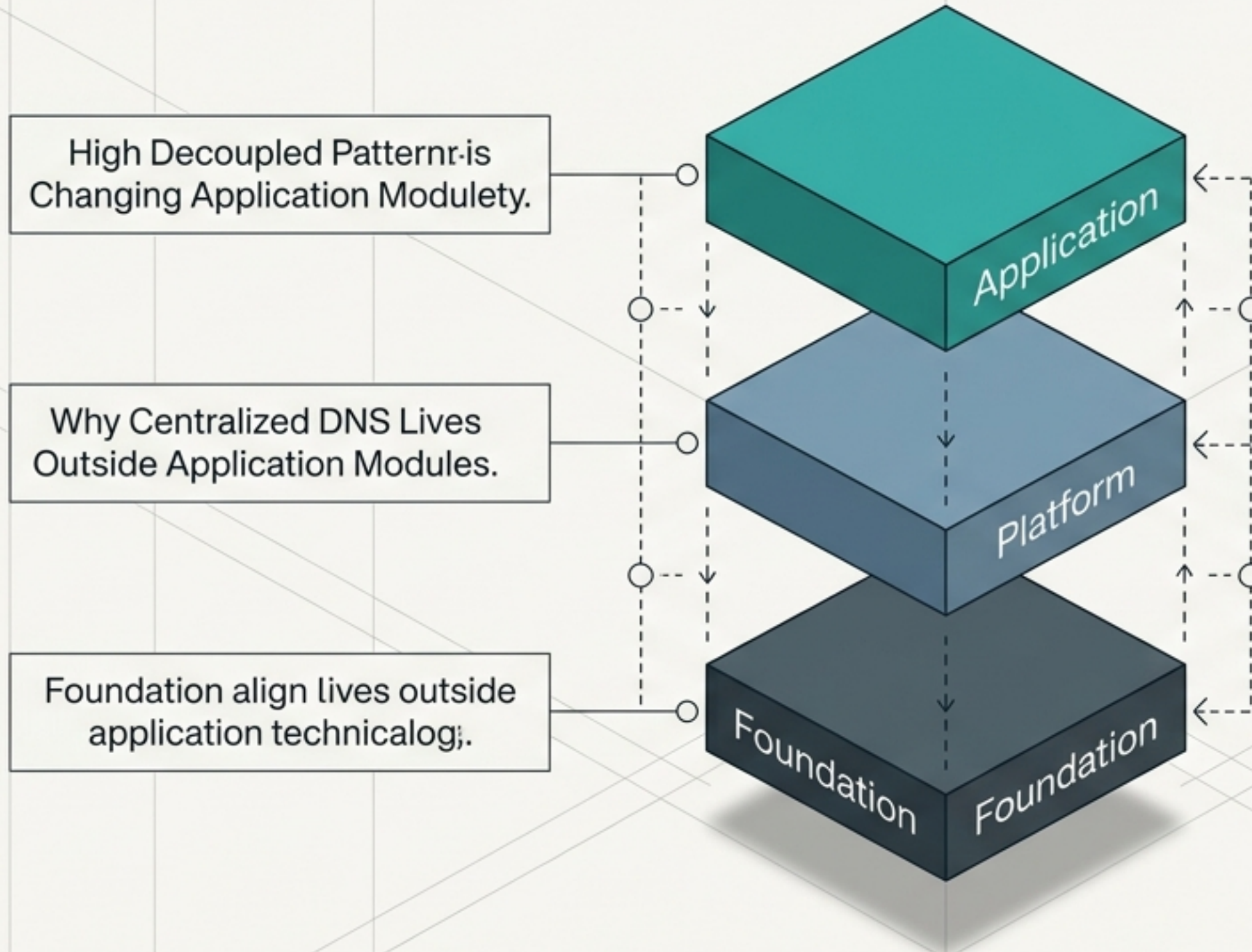


# Vision 2026 Enterprise Architecture Patterns

## The Decoupled Foundation Why Centralized DNS Lives Outside Application Modules

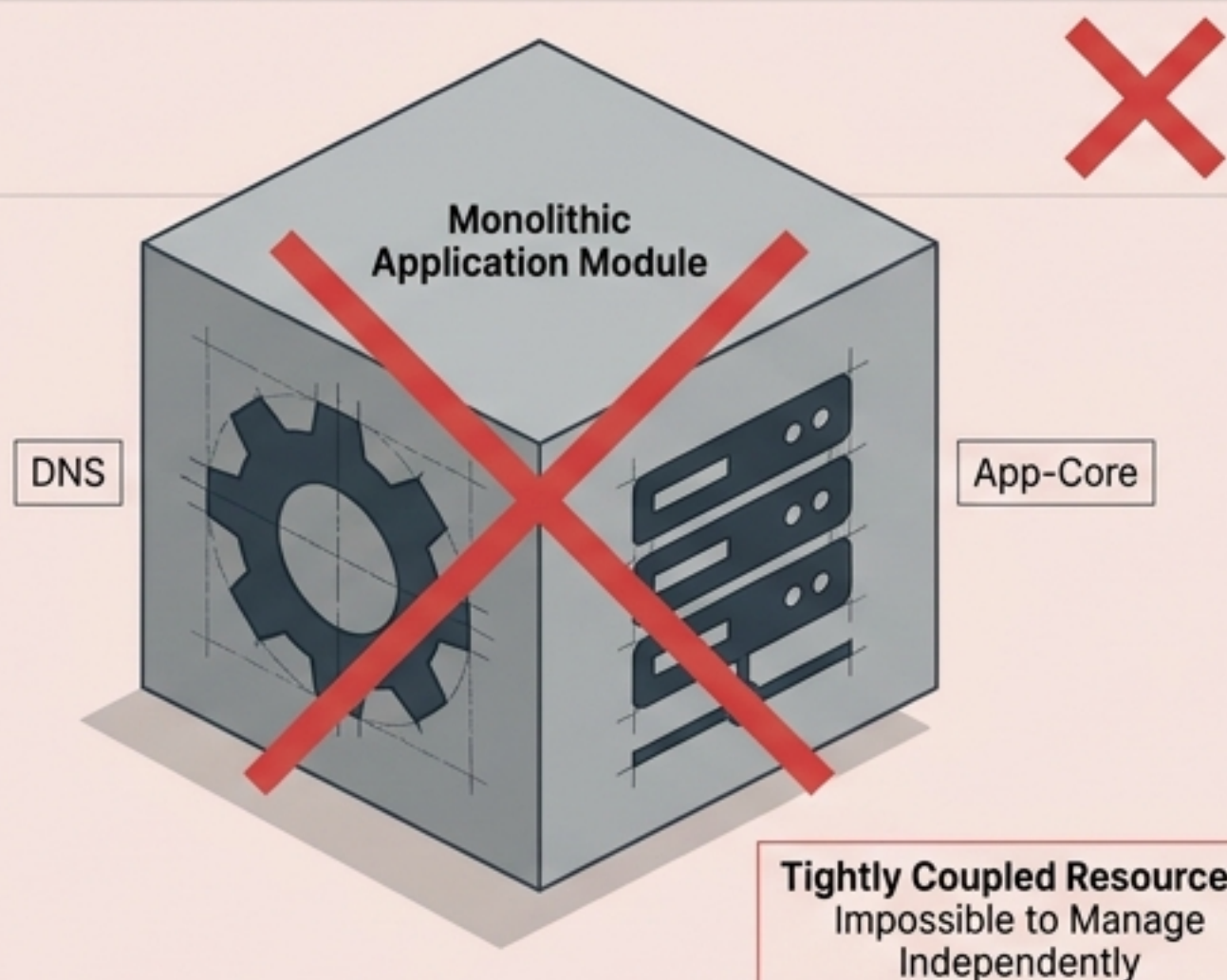


# The Architectural Dilemma

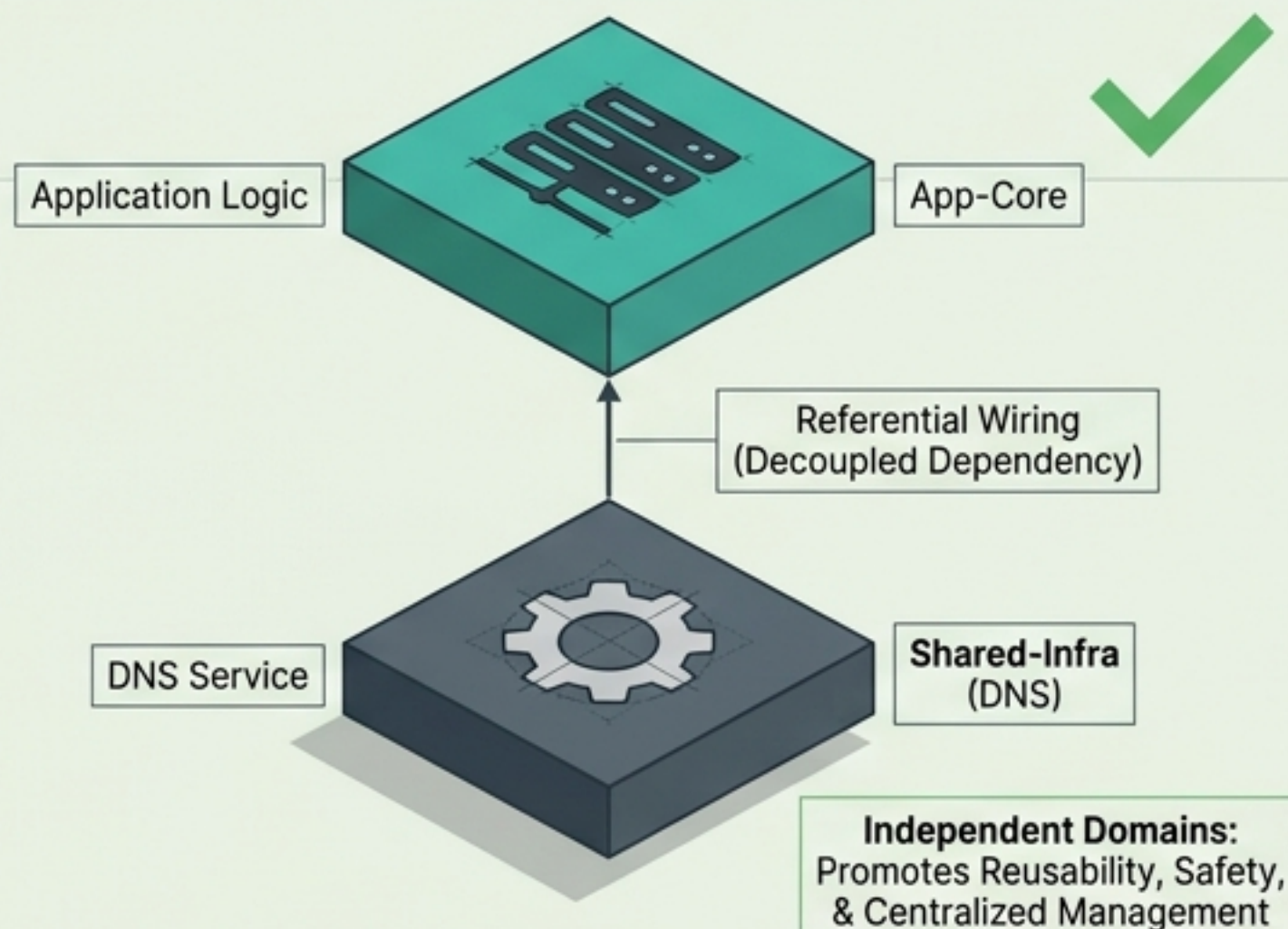
Why not bake the `dns_top_level_domain_module` directly into App-Core or App-Catalog?

In a mono-repo, bundling everything an app needs into a single Terraform module seems convenient. However, enterprise-scale infrastructure demands Domain-Driven Composition to ensure safety, reusability, and centralized networking.

## The Monolith (Coupled)

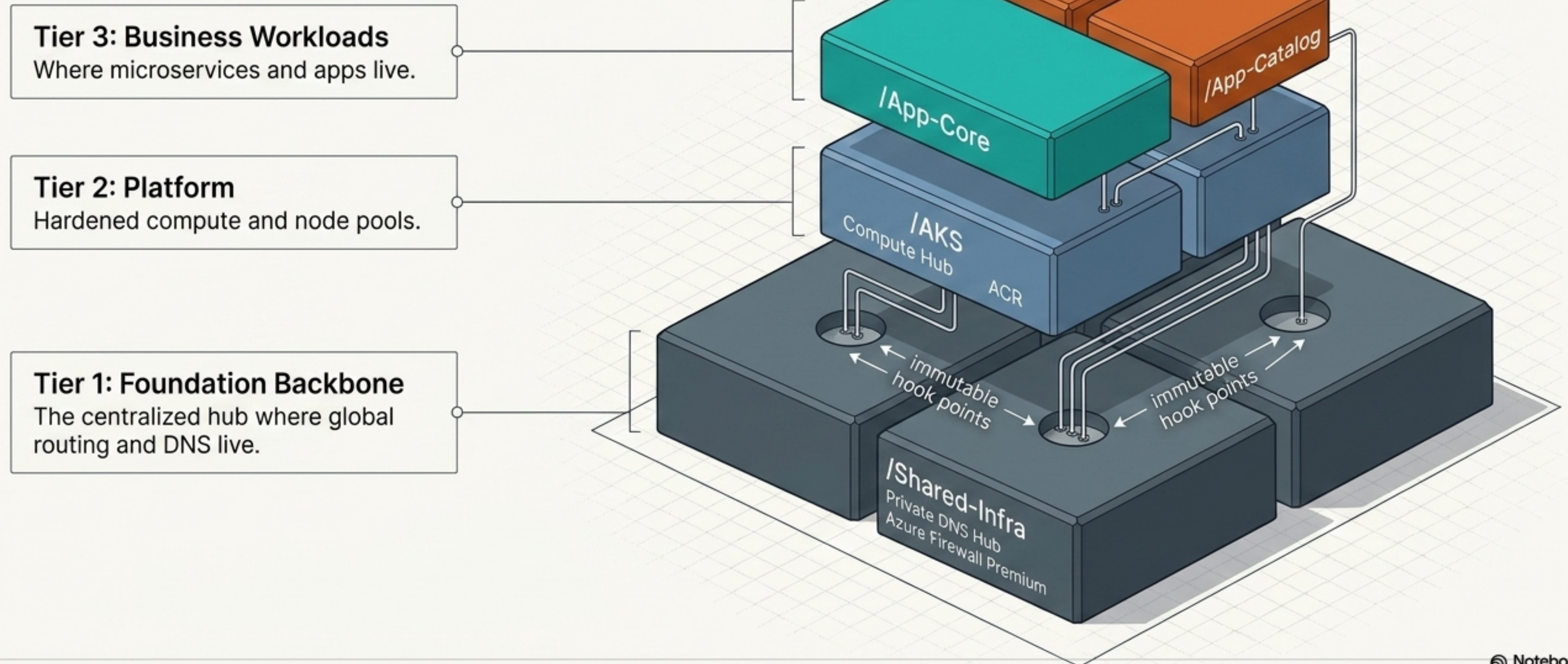


## Domain-Driven (Decoupled)



# Infrastructure as a Layered Stack

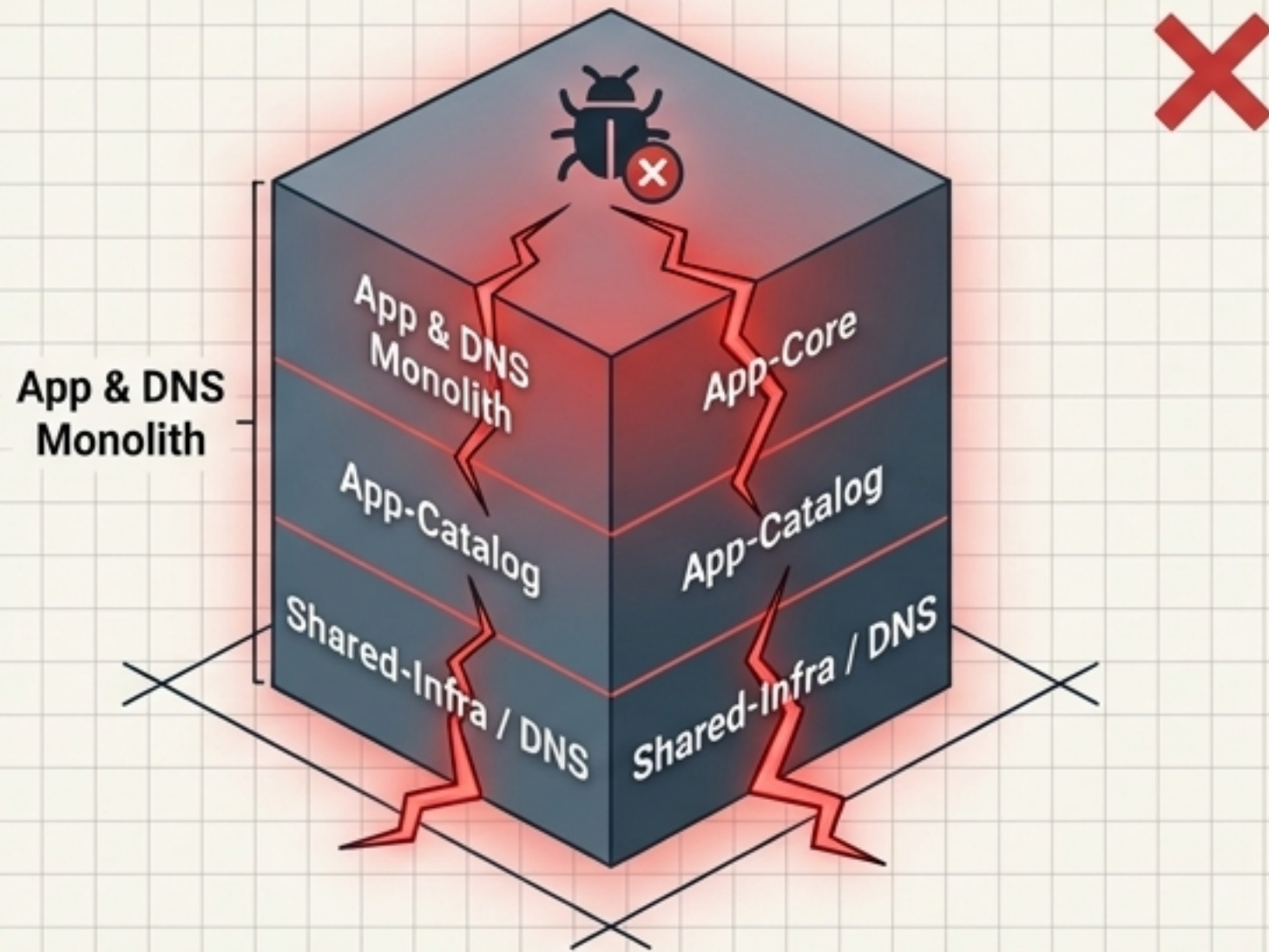
**Key Insight:** Each tier provides immutable hook points for the layer above it. Apps consume DNS; they do not own it.



# Containing the Blast Radius

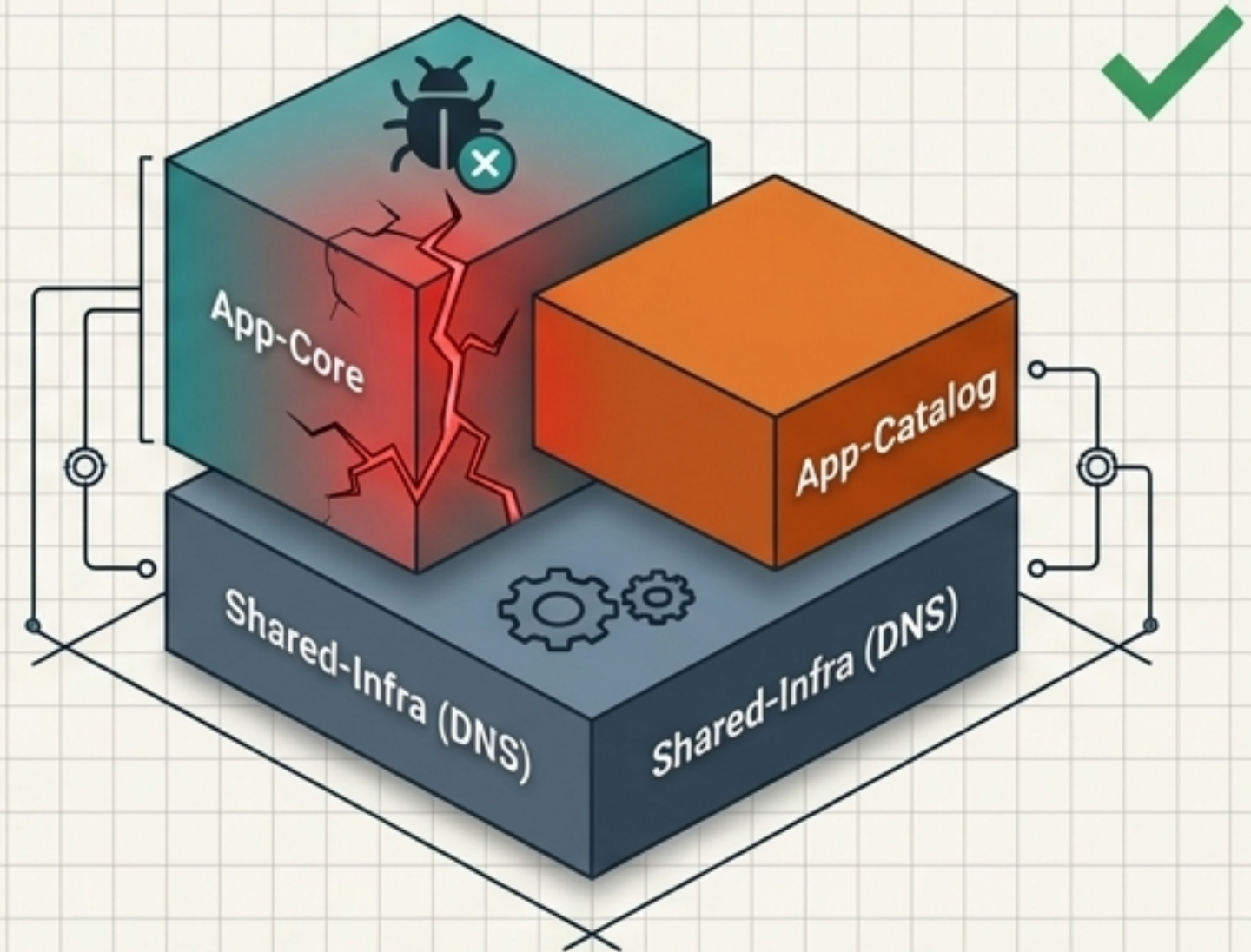
Using isolated Terraform states (`terraform.tfstate`) per logical domain ensures that application-level provisioning errors cannot corrupt or lock foundational enterprise routing.

## A Coupled: Global Failure



Bug in App code corrupts shared state, destroying DNS.

## B Decoupled: Localized Failure



App state locks, but foundational DNS backbone remains unaffected.

# Decoupled Lifecycles

**Applications mutate daily. Core enterprise DNS mutates rarely.**

Partial Lifecycle Management allows engineering teams to perform aggressive maintenance on App-Core without risking state locks or downtime on the Shared-Infra backbone.

## Tier 3: App-Core (Business Workloads)



## Tier 1: Shared-Infra DNS (Foundation)

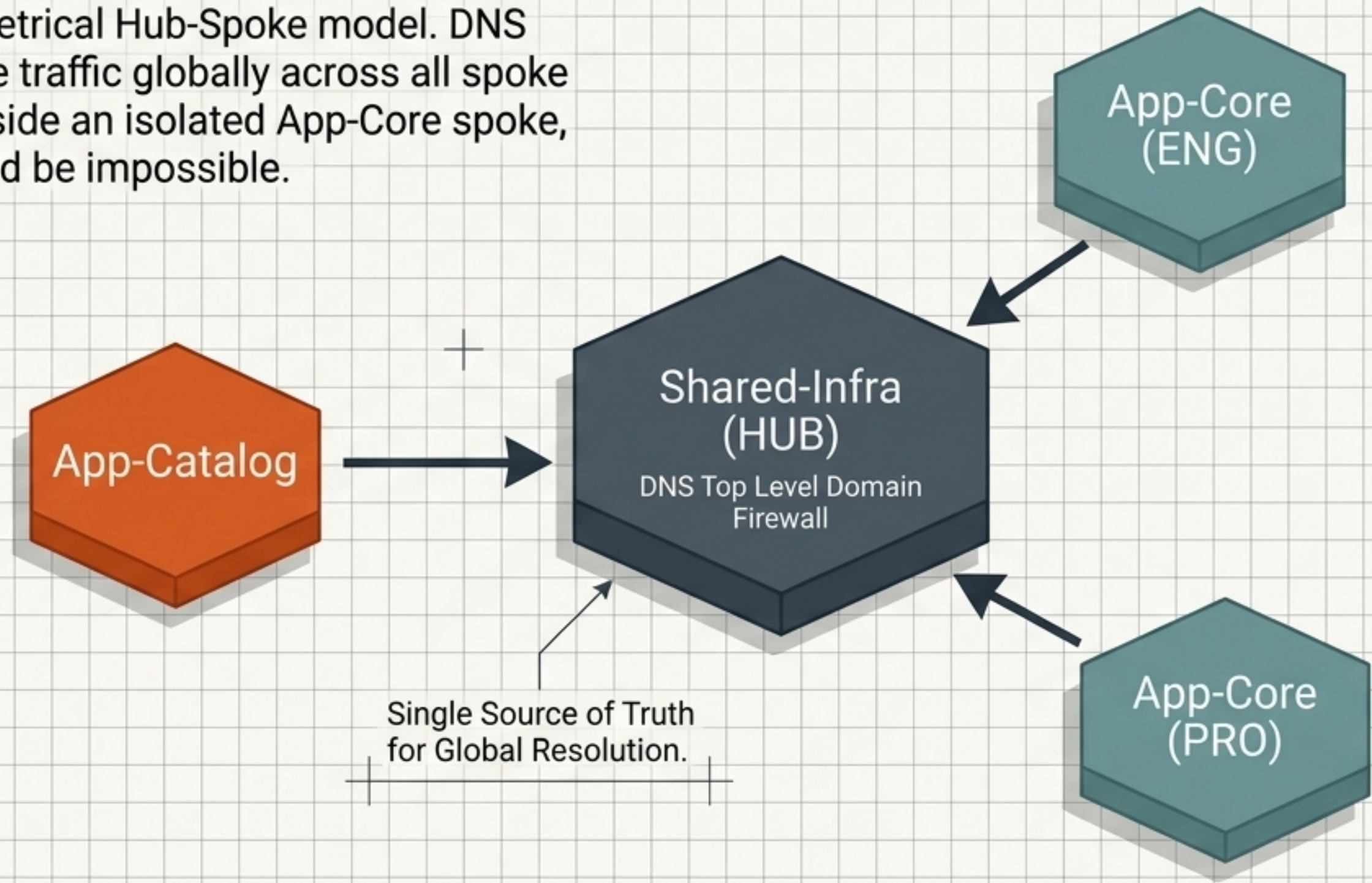


**Tier 3 (App-Core):** Vibrant Teal

**Tier 1 (Shared-Infra):** Deep Slate Grey

# Centralized Routing, Decentralized Apps

Vision 2026 relies on a Symmetrical Hub-Spoke model. DNS must live in the Hub to resolve traffic globally across all spoke environments. If DNS lived inside an isolated App-Core spoke, cross-module resolution would be impossible.



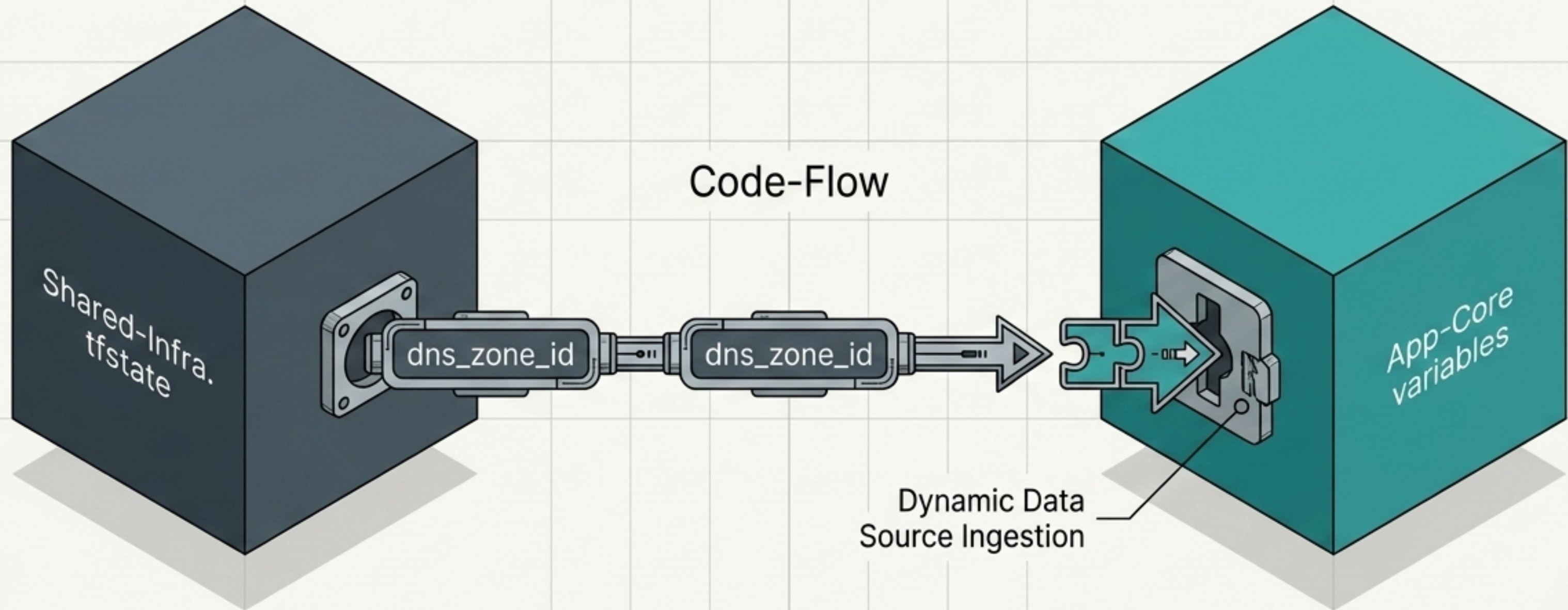
# Architectural Trade-Offs

| Dimension             | Coupled DNS (Atomic)          | Decoupled DNS (Domain-Driven)                             |
|-----------------------|-------------------------------|-----------------------------------------------------------|
| State Lock Contention | High (Teams block each other) | <b>Zero</b> (Independent stack states)                    |
| Reusability           | Siloed to specific apps       | Shared globally across App-Core, Catalog, and Integration |
| Security & RBAC       | All-or-Nothing access         | Fine-grained Identity Governance                          |
| Blast Radius          | Global                        | Localized to specific app spoke                           |

# Connecting the Layers via Referential Linking

We strictly avoid hardcoding. Instead, we use Referential Attribute Linking to maintain 100% IaC Sovereignty.

Downstream stacks (App-Core) consume outputs from upstream stacks (Shared-Infra) dynamically via Data Sources or Azure DevOps Pipeline Variables, ensuring identical environments.

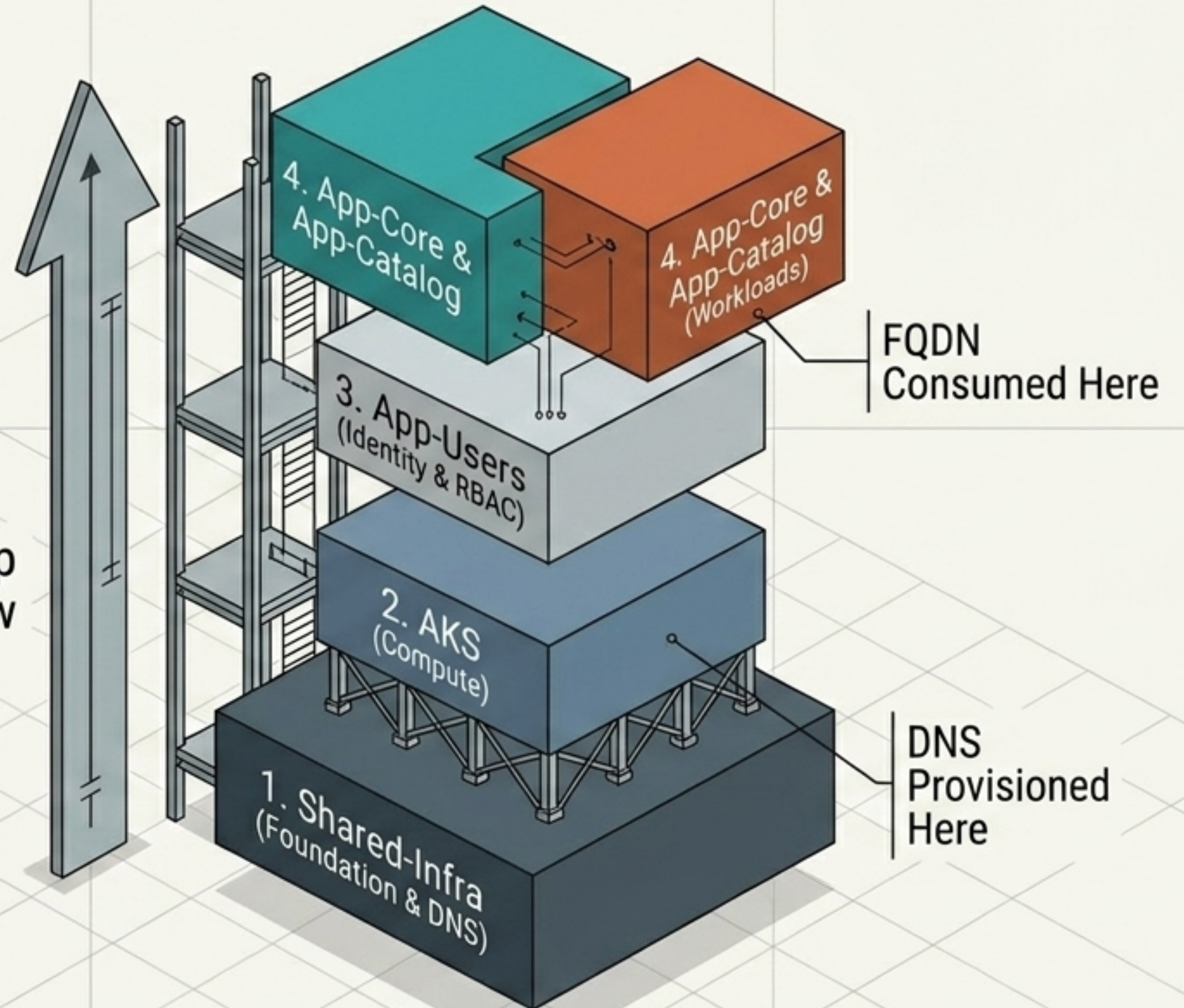


# The Plan-First Orchestration Sequence

Infrastructure must be provisioned from the bottom up to satisfy network dependencies.

The `dns_top_level_domain_module` must exist before App-Core can request FQDNs or attach Application Gateways.

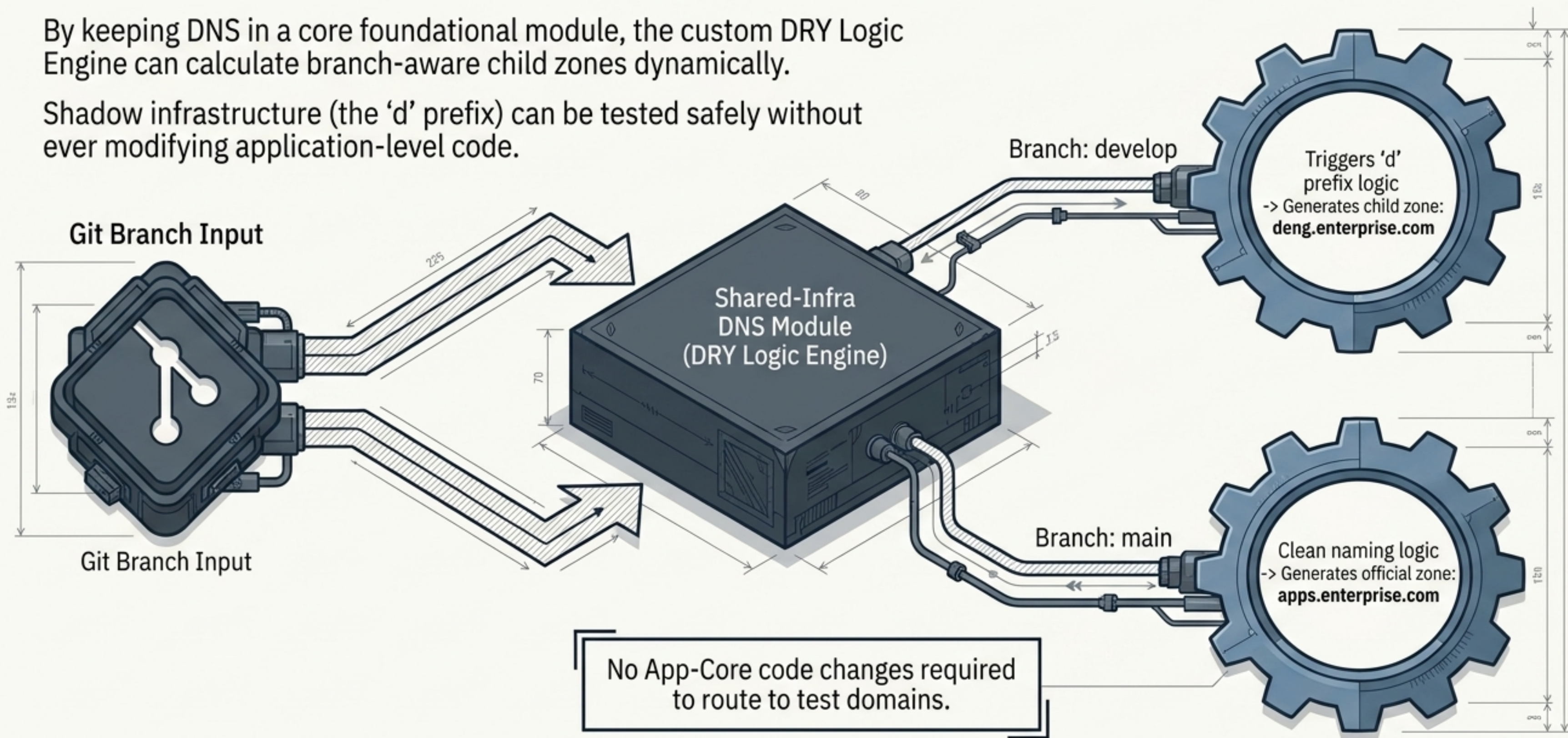
Strict Bottom-Up  
Provisioning Flow



# Centralized Logic for Dynamic FQDNs

By keeping DNS in a core foundational module, the custom DRY Logic Engine can calculate branch-aware child zones dynamically.

Shadow infrastructure (the 'd' prefix) can be tested safely without ever modifying application-level code.



# The Power of Domain-Driven Composition

Decoupling DNS from App-Core is not merely an organizational preference. It is the foundational requirement for Hub-Spoke scaling, absolute Blast Radius containment, and 100% Terraform IaC Sovereignty.

